Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

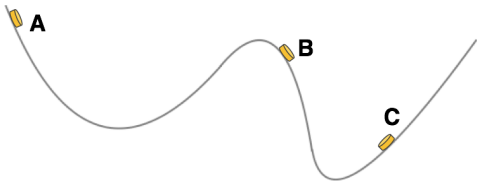
Advanced Training Methods: Momentum

A Physical Perspective

- Consider the movement of a ball going downhill
- The ball will eventually stop somewhere with locally low potential energy
- The trajectory is described by Newton's law of motion $F = m\ddot{w}$
- The ball experiences two forces: a force due to the potential energy $-\nabla f(w)$ and a drag force $-\rho\dot{w}$. Thus

$$m\ddot{w} = -\rho\dot{w} - \nabla f(w), \quad \equiv \quad \begin{cases} \dot{p} = -\alpha\frac{p}{m} - \nabla f(w) \\ \dot{w} = \frac{p}{m} \end{cases} \tag{1}$$

- Thus, conceptually, depending on the built up momentum p and the strength of the drag force, the ball could reside at a point with the globally lowest potential.



Deriving GD



- Let us consider the case where the ball is a massless particle. Assuming its speed is less than speed of light,

$$\dot{w} = -\rho^{-1}\nabla f(w) \qquad \text{called gradient flow} \tag{2}$$

- Using Euler's discretization

$$w_{t+1} = w_t - \rho^{-1}\nabla f(w_t) \tag{3}$$

  which is exactly our GD method
- Thus, GD shows the trajectory of massless or light ball
- Here, the momentum is zero and the ball may not be able to avoid getting stuck in a locally low potential state

Deriving the Heavy Ball Method



- Let us now consider the heavy ball case. using Euler's discretization again (and after basic manipulation and renaming the constants) we can show

$$w_{t+1} = w_t - \eta \nabla f(w_t) + \beta(w_t - w_{t-1}) \tag{4}$$

or equivalently

$$\begin{cases} w_{t+1} = w_t - \eta p_t \\ p_t = \beta p_{t-1} + \nabla f(w_t) \end{cases} \tag{5}$$

- This method is called the Heavy Ball Method with momentum parameter $0 \leq \beta < 1$ and serves a a key ingredient in DL optimization
- The correction term could aid reaching higher quality solutions compared to simple GD

Nesterov's Accelerated Gradient Method



- Assuming a time-varying drag force $-\frac{\rho}{t}\dot{w}$ the law becomes

$$\ddot{w} + \frac{\rho}{t}\dot{w} + \nabla f(w) = 0 \tag{6}$$

- It turns our this ODE is intimately related to another method called Nesterov's Accelerated Gradient (NAG)

$$\begin{cases} w_{t+1} = w_t - \eta p_t \\ p_t = \beta p_{t-1} + \nabla f(w_t - \eta\beta p_t) \end{cases} \tag{7}$$

- Called accelerated since it achieves an accelerated (and optimal) convergence rate compared to GD for smooth and strongly convex functions.
- NAG and other accelerated variants of GD can also be thought of as better discretization of our original law of motion, e.g., applying two-point Euler's method.

Momentum in DL



- In the context of DL optimization, we typically use m_t instead of p_t and also use the following variant

$$\begin{cases} w_{t+1} = w_t - \eta m_t \\ m_t = \beta m_{t-1} + (1-\beta) g_t, \quad m_0 = 0 \end{cases} \tag{8}$$

  which we call SGD with momentum (SGDm)

- Essentially, the momentum performs an exponential moving average on all stochastic gradients observed so far and uses that in the update

$$m_t = (1-\beta) \sum_{k=0}^{t-1} \beta^k g_{t-k} \tag{9}$$

Momentum as a new Gradient Estimator



- Recall under the SFO assumption, g_t is (conditionally) an unbiased estimator of $\nabla f(w_t)$ with variance σ^2
- In SGDm, we use $m_t = \beta m_{t-1} + (1-\beta)g_t$, $m_0 = 0$ instead of g_t to update our parameter
- Note that from an estimation perspective, this leads to bias as

$$\begin{aligned}\mathbb{E}_{z_t}[m_t|z_{1:t-1}] &= \mathbb{E}_{z_t}[\beta m_{t-1} + (1-\beta)g_t|z_{1:t-1}] \\ &= \beta\mathbb{E}_{z_t}[m_{t-1}|z_{1:t-1}] + (1-\beta)\mathbb{E}_{z_t}[g_t|z_{1:t-1}] \\ &= \beta\mathbb{E}_{z_t}[m_{t-1}|z_{1:t-1}] + (1-\beta)\nabla f(w_t) \neq \nabla f(w_t)\end{aligned} \tag{10}$$

- But, perhaps we are gaining some reduction in the variance σ^2 and as a result a reduction in total estimation error



- Recalling $m_0 = 0$ and expanding $m_t = \beta m_{t-1} + (1-\beta) g_t$ over time

$$m_t = (1-\beta) \sum_{k=0}^{t-1} \beta^k g_{t-k} \tag{11}$$

- To gain intuition, let us assume each g_{t-k} is an independent unbiased estimator $\nabla f(w_t)$ with variance $\mathbb{E}\|g_{t-k} - \nabla f(w_t)\|^2 \leq \sigma^2$. Our first take away is that

$$\mathbb{E}[m_t] = \nabla f(w_t) \cdot (1-\beta) \cdot \sum_{k=0}^{t-1} \beta^k = \nabla f(w_t) \cdot (1-\beta) \cdot \frac{(1-\beta^t)}{(1-\beta)} = \nabla f(w_t) \cdot (1-\beta^t) \tag{12}$$

so we have a bias that is vanishing as we train our model and $t \to \infty$. This observation provides a way to "de-bias" the momentum by dividing with $(1-\beta^t)$

$$m_t = \beta m_{t-1} + (1-\beta) g_t, \quad \hat{m}_t = m_t/(1-\beta^t), \quad w_{t+1} = w_t - \eta \hat{m}_t \tag{13}$$

- Let us now consider the variance of de-biased momentum $\hat{m}_t$ under our simple setting

$$\begin{aligned}
\mathbb{E}\|\hat{m}_t - \nabla f(w_t)\|^2 &= \left\| \frac{(1-\beta)}{(1-\beta^t)} \sum_{k=0}^{t-1} \beta^k g_{t-k} - \nabla f(w_t) \right\|^2 \\
&= \mathbb{E} \left\| \frac{(1-\beta)}{(1-\beta^t)} \sum_{k=0}^{t-1} \beta^k [g_{t-k} - \nabla f(w_t)] \right\|^2 \\
&= \frac{(1-\beta)^2}{(1-\beta^t)^2} \sum_{k=0}^{t-1} \beta^{2k} \mathbb{E}\|g_{t-k} - \nabla f(w_t)\|^2 \\
&\leq \sigma^2 \frac{(1-\beta)^2}{(1-\beta^t)^2} \sum_{k=0}^{t-1} \beta^{2k} = \sigma^2 \underbrace{\frac{1-\beta^{2t-1}}{(1-\beta^t)^2}}_{:=B(t)} \frac{(1-\beta)^2}{1-\beta^2} = \sigma^2 \frac{1-\beta}{1+\beta} B(t)
\end{aligned} \tag{14}$$

Implicit Mini Batching (Cont'd)



- $B(t) = \frac{1-\beta^{\mathbf{2}t-\mathbf{1}}}{(\mathbf{1}-\beta^t)^{\mathbf{2}}}$ is a decreasing function of $t \geq 1$ and approaches to 1 as $t \to \infty$
- Thus, we have shown as we train the variance of de-biased momentum decreases
- Let us now assume $B(t) \approx 1$ and we use a large momentum parameter like $\beta = 0.9$

$$\mathbb{E}\|\hat{m}_t - \nabla f(w_t)\|^2 \approx B(t)\frac{1-\beta}{1+\beta}\sigma^2 \approx \frac{1-\beta}{1+\beta}\sigma^2 \approx \frac{1-0.9}{1+0.9}\sigma^2 \approx \frac{\sigma^2}{20} \tag{15}$$

  effectively a mini-batch SGD with batch size 20.

- Important Remark: Our intuitive analysis is not precise and even the de-biased momentum suffers from some bias. We will develop a formal study soon.

The ADAM optimizer



ADAM, leverages theses four component into an effective method: momentum, de-basing, adaptive η, per coordinate η (and optionally weight-decay):

- $z_t \sim p_z$ (Generating a mini batch)
- $g_t = \nabla \ell(w_t, z_t) + \lambda w_t$ (compute the stochastic gradient)
- $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ (compute the momentum)
- $\hat{m}_t = m_t / (1-\beta_1^t)$ (de-bias the momentum)
- $S_t(i) = \beta_2 S_{t-1}(i) + (1-\beta_2)|g_t(i)|^2$ (EMA for adaptive per coordinate variance estimation)
- $\hat{S}_t(i) = S_t(i)/(1-\beta_2^t)$ (de-bias the variance)
- $\eta_t(i) = \tilde{\eta}_t(i) / \sqrt{\kappa^2 + \hat{S}_t(i)}$ (adaptive per coordinate learning rate)
- $w_{t+1}(i) = [1 - \lambda \eta_t(i)] w_t(i) - \eta_t(i) \hat{m}_t(i)$ (parameter update)